

Unit II

PROCESS MANAGEMENT

Course Objective: To analyze scheduling algorithms and process synchronization.

Course Outcome: Analyze and evaluate CPU scheduling algorithms and assess the solutions for synchronization problems and dead lock prevention

Part A

S.No.	Question	BTL *
1.	Recall Process. A Process can be thought of as a program in execution. A process will need certain resources such as CPU time, memory, files & I/O devices to accomplish its task	K1
2.	Illustrate the process states with a diagram. • New: The process is being created. • Running: Instruction is being executed. • Waiting: The process is waiting for some event to occur. • Ready: The process is waiting to be assigned to a processor. • Terminated: The process has finished execution.	K2
3.	State process control block. List out the data field associated with PCB. Each process is represented in the operating system by a process control block also called a task control block. • Process state • Process number • Program counter • CPU registers • Memory limits • List of open files • CPU scheduling information • Memory management information	K1

	- Accounting information - I/O status information	
4.	Recall the critical section problem. - The critical section is a part of the program where a process accesses shared resources like files, memory, or variables. - If multiple processes enter their critical sections at the same time, it can cause data inconsistency and race conditions.	K1
5.	Distinguish between cooperating processes and independent processes. Independent process: □ A process is independent if it cannot affect or be affected by the other processes executing in the system. A process that does not share data with any other process is independent. Co operating process: □ A process is co-operating if it can affect or be affected by other processes executing in the system. Any process that shares data with any other process is cooperating.	K2
6.	List the types of process schedulers. Long-term scheduler: Decides which processes should enter the ready queue from the job queue. Short-term scheduler: Chooses which process will execute next on the CPU. Medium-term scheduler: Temporarily removes processes from memory to optimize CPU and memory usage (also called swapping).	K1
7.	State the purpose of a mutex in process synchronization. A mutex (mutual exclusion lock) is a synchronization mechanism ensuring that only one process can access a critical section at a time. If a process locks a mutex, other processes must wait until it is released before accessing the shared resource.	K1
8.	Distinguish between short-term scheduler and long-term scheduler. The primary distinction between the two schedulers lies in the frequency of execution. - The short term scheduler is designed to frequently select a new process for the CPU, at least once every 100 milliseconds. Because of the short time between executions, the short-term scheduler must be fast. - The long-term scheduler executes much less frequently; minutes may separate the creation of one new process and the next. The long-term scheduler controls the degree of multiprogramming. Because of the longer interval between executions, the long-term scheduler can afford to take more time to decide which process should be selected for execution.	K2
9.	List the various scheduling criteria for CPU scheduling. The various scheduling criteria are - CPU utilization - Throughput - Turnaround time - Waiting time - Response time	K1

10.	Compare busy waiting and a spinlock. Busy waiting: When a process is in its critical section, any other process that tries to enter its critical section Must loop continuously in the entry code. This is called as busy waiting. Spinlock: Busy waiting waster CPU cycles that some other process might be able to use productively. This type of semaphore is also called a spinlock because the process “spin” while waiting for the lock.	K2
11.	Write the pseudocode for the TestAndSet() instruction. <pre>boolean TestAndSet (boolean &target) { boolean rv=*target; *target=true; return rv; }</pre>	K2
12.	State the purpose of Inter-Process Communication (IPC). - IPC is a mechanism that allows independent processes to exchange data and signals. - It helps processes coordinate and share resources, enabling multi-process execution without data inconsistency.	K1
13.	Name two IPC mechanisms. Shared Memory: Processes share a common memory space for communication, but they need synchronization mechanisms like semaphores. Message Passing: Processes communicate using send() and receive() messages via OS-controlled message queues.	K1
14.	Recall mutual exclusion. Mutual exclusion refers to the requirement of ensuring that no two process or threads are in their critical section at the same time. i.e. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.	K1
15.	State semaphore and its importance in an operating system. A semaphore 'S' is a synchronization tool which is an integer value that, apart from initialization, is accessed only through two standard atomic operations; wait and signal. Semaphores can be used to deal with the n-process critical section problem. It can be also used to solve various Synchronization problems.	K1
16.	Distinguish between preemptive and non-preemptive scheduling. Non-preemptive scheduling once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or switching to the waiting state. Preemptive scheduling can preempt a process which is utilizing the CPU in between its execution and give the CPU to another process.	K2

17.	Recall turn around time. Turn around time is the interval from the time of submission to the time of completion of a process. It is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.	K1
18.	List four situations where CPU scheduling decisions occur. CPU scheduling decisions take place under one of four conditions: • When a process switches from the running state to the waiting state, such as for an I/O request or invocation of the wait() system call. • When a process switches from the running state to the ready state, for example in response to an interrupt. • When a process switches from the waiting state to the ready state, say at completion of I/O or a return from wait(). • When a process terminates.	K1
19.	Recall deadlock. A process requests resources; if the resources are not available at that time, the process enters a wait state. Waiting processes may never again change state, because the resources they have requested are held by other waiting processes. This situation is called a deadlock.	K1
20.	List the necessary conditions for deadlock. A deadlock situation can arise if the following four conditions hold simultaneously in a system: a. Mutual exclusion b. Hold and wait c. No pre-emption d. Circular wait	K1
21	List the methods for handling deadlocks. The deadlock problem can be dealt with in one of the three ways: 1) Use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlock state. 2) Allow the system to enter the deadlock state, detect it and then recover. 3) Ignore the problem all together, and pretend that deadlocks never occur in the system.	K1

Part B

S.No.	Question	BTL *																								
1.	Explain the Interprocess Communication. (16)	K2																								
2.	Analyze the following set of processes with the length of the CPU burst time given in milliseconds. (16) <table><tr><td>Process</td><td>Burst Time</td><td>Priority</td></tr><tr><td>P1</td><td>10</td><td>3</td></tr><tr><td>P2</td><td>1</td><td>1</td></tr><tr><td>P3</td><td>2</td><td>3</td></tr><tr><td>P4</td><td>1</td><td>4</td></tr><tr><td>P5</td><td>5</td><td>2</td></tr></table> The processes are assumed to have arrived in the order P1, P2, P3, P4, P5 all at time 0. a. Draw four Gantt charts illustrating the execution of these processes using FCFS, SJF, a non-pre-emptive priority (a smaller priority number implies a higher priority) and RR (quantum = 1) scheduling. b. Determine the turnaround time of each process for each of the scheduling algorithms in part (a)? c. Determine the waiting time of each process for each of the scheduling algorithms in part (a)? d. Which of the schedules in part (a) results in the minimal average waiting time?	Process	Burst Time	Priority	P1	10	3	P2	1	1	P3	2	3	P4	1	4	P5	5	2	K3						
Process	Burst Time	Priority																								
P1	10	3																								
P2	1	1																								
P3	2	3																								
P4	1	4																								
P5	5	2																								
3.	Analyze the following set of processes with the length of CPU-burst time given in milliseconds. <table><tr><td>Process</td><td>Burst time</td><td>Priority</td><td>Arrival time</td></tr><tr><td>P₁</td><td>10</td><td>3</td><td>0</td></tr><tr><td>P₂</td><td>1</td><td>1</td><td>1</td></tr><tr><td>P₃</td><td>2</td><td>3</td><td>2</td></tr><tr><td>P₄</td><td>1</td><td>4</td><td>1</td></tr><tr><td>P₅</td><td>5</td><td>2</td><td>2</td></tr></table> Draw the Gantt chart for the execution of these processes using FCFS, SJF, SRTS, pre-emptive and non-pre-emptive priority, round robin with time slice of 2 ms. Calculate the average waiting and turnaround time using each of the methods. (16)	Process	Burst time	Priority	Arrival time	P ₁	10	3	0	P ₂	1	1	1	P ₃	2	3	2	P ₄	1	4	1	P ₅	5	2	2	K3
Process	Burst time	Priority	Arrival time																							
P ₁	10	3	0																							
P ₂	1	1	1																							
P ₃	2	3	2																							
P ₄	1	4	1																							
P ₅	5	2	2																							

4.	Analyze the following snapshot of a system. (16)	K3																																																																																										
<table><tr><th rowspan="2">Processes</th><th colspan="4">Allocation</th><th colspan="4">Max</th><th colspan="4">Available</th></tr><tr><th>A</th><th>B</th><th>C</th><th>D</th><th>A</th><th>B</th><th>C</th><th>D</th><th>A</th><th>B</th><th>C</th><th>D</th></tr><tr><td>P₀</td><td>2</td><td>0</td><td>0</td><td>1</td><td>4</td><td>2</td><td>1</td><td>2</td><td>3</td><td>3</td><td>2</td><td>1</td></tr><tr><td>P₁</td><td>3</td><td>1</td><td>2</td><td>1</td><td>5</td><td>2</td><td>5</td><td>2</td><td></td><td></td><td></td><td></td></tr><tr><td>P₂</td><td>2</td><td>1</td><td>0</td><td>3</td><td>2</td><td>3</td><td>1</td><td>6</td><td></td><td></td><td></td><td></td></tr><tr><td>P₃</td><td>1</td><td>3</td><td>1</td><td>2</td><td>1</td><td>4</td><td>2</td><td>4</td><td></td><td></td><td></td><td></td></tr><tr><td>P₄</td><td>1</td><td>4</td><td>3</td><td>2</td><td>3</td><td>6</td><td>6</td><td>5</td><td></td><td></td><td></td><td></td></tr></table>			Processes	Allocation				Max				Available				A	B	C	D	A	B	C	D	A	B	C	D	P ₀	2	0	0	1	4	2	1	2	3	3	2	1	P ₁	3	1	2	1	5	2	5	2					P ₂	2	1	0	3	2	3	1	6					P ₃	1	3	1	2	1	4	2	4					P ₄	1	4	3	2	3	6	6	5				
Processes	Allocation				Max				Available																																																																																			
	A	B	C	D	A	B	C	D	A	B	C	D																																																																																
P ₀	2	0	0	1	4	2	1	2	3	3	2	1																																																																																
P ₁	3	1	2	1	5	2	5	2																																																																																				
P ₂	2	1	0	3	2	3	1	6																																																																																				
P ₃	1	3	1	2	1	4	2	4																																																																																				
P ₄	1	4	3	2	3	6	6	5																																																																																				
Answer the following questions using Banker's algorithm.																																																																																												
i. Illustrate that the system is in safe state by demonstrating an order in which the processes may complete? ii. If a request from a process P₁ arrives for (1, 1, 0, 0), can the request be granted immediately? iii. If the request from P₄ arrives for (0, 0, 2, 0), can the request be granted immediately?																																																																																												
5.	State critical section problem. Discuss three solutions to solve the critical section problem. (16)	K2																																																																																										
6.	Outline a solution using Semaphores to solve Dining Philosopher problem. (8)	K3																																																																																										
7.	Explain deadlock prevention and deadlock avoidance algorithms. (16)	K2																																																																																										
8.	Describe deadlock detection and recovery from deadlock. (8)	K2																																																																																										
9.	Explain producer consumer problem with an example. (8)	K2																																																																																										
10.	Case Study: "GreenCity" Traffic Control System Scenario: GreenCity is implementing a smart traffic management system for a metropolitan area. The system uses a network of sensors at each intersection and a central control server running multiple software processes. Each intersection is managed by an Intersection Controller Process (ICP). Vehicles are detected by sensors, and the system must coordinate traffic lights to optimize flow while ensuring safety. Key System Processes: 1. Sensor Data Processor (SDP): Reads raw sensor data and detects vehicle presence. 2. Traffic Light Controller (TLC): Controls the actual traffic lights. 3. Central Coordinator (CC): Makes high-level decisions for multiple intersections. 4. Emergency Vehicle Handler (EVH): Gives priority to emergency vehicles. System Challenges: • Multiple ICPs sometimes try to access a shared "congestion map" in memory simultaneously. • The system must avoid situations where: ○ Two perpendicular directions get a green light simultaneously (critical safety violation). ○ An emergency vehicle is stuck waiting indefinitely while other processes hold resources. • The CPU scheduler on the central server must handle these processes efficiently.	K3																																																																																										

	Based on the case study, answer the following: - Analyze the synchronization requirements between the Sensor Data Processor (SDP) and Traffic Light Controller (TLC) processes when updating and reading the traffic light status. Propose a solution using semaphores to solve the critical section problem in this context, ensuring no conflicting light commands occur. (8) - Suppose the system encounters a situation where: <i>Process A</i> (Emergency Vehicle Handler) holds the "north-south green light" resource and requests the "central database lock." <i>Process B</i> (Central Coordinator) holds the "central database lock" and requests the "east-west green light" resource. <i>Process C</i> (Traffic Light Controller) holds the "east-west green light" resource and requests the "north-south green light" resource. Identify what problem this represents. Apply a deadlock handling strategy (prevention, avoidance, detection, or recovery) most suitable for this real-time traffic system, justifying your choice with specific techniques from your selected strategy. (8)	
--	---	--